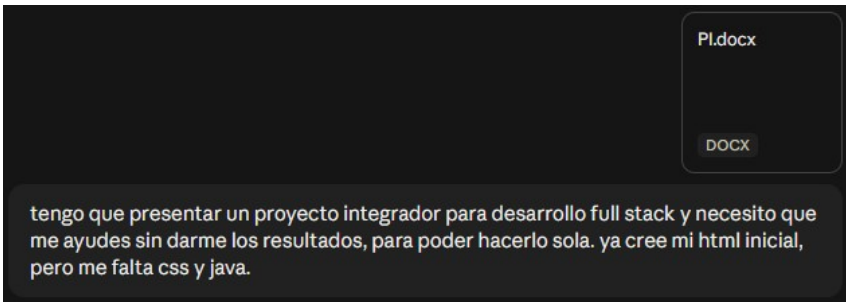
REGISTRO CRONOLOGICO DEL USO IA E INTERVENCIONES

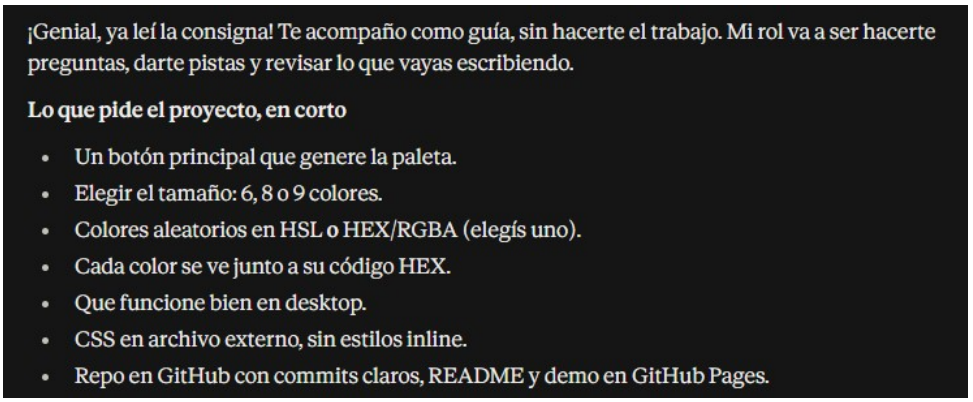
Consulta: Le solicito a Claude una guía para preparar mi proyecto integrador, sin hacerme el trabajo, proporcionándole mi index.html

Prompts:



Respuesta de la IA:

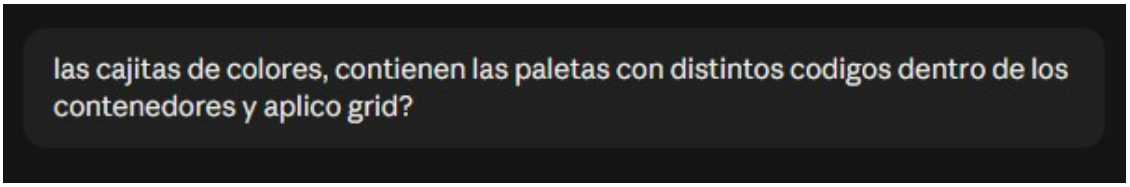
Hoja de ruta y preguntas. **Sin código.**



Lo que decidí o cambié yo: comenzar con CSS.

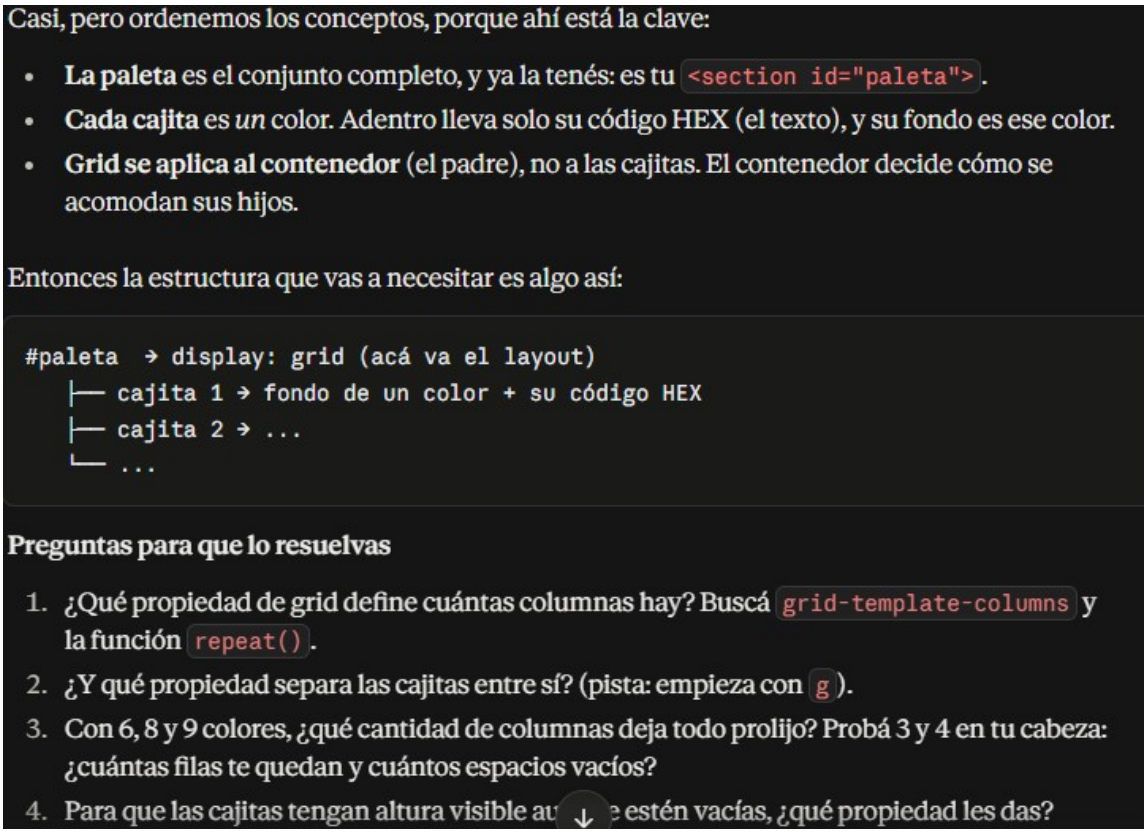
Consulta: Le pido a Claude que revise mis códigos de paleta de colores, porque no puedo verlos, sin pedirle resultados de cómo hacerlo, me da preguntas para pensar para analizar que estoy haciendo mal, sigo probando… mientras tengo abierta mi lectura de CSS para seguir repasando conceptos.

Prompts:



Respuesta de la IA:

Explicación de contenedor y cajitas, con grid, repeat y gap.



Casi, pero ordenemos los conceptos, porque ahí está la clave:

- **La paleta** es el conjunto completo, y ya la tenés: es tu `<section id="paleta">`.
- **Cada cajita** es *un* color. Adentro lleva solo su código HEX (el texto), y su fondo es ese color.
- **Grid se aplica al contenedor** (el padre), no a las cajitas. El contenedor decide cómo se acomodan sus hijos.

Entonces la estructura que vas a necesitar es algo así:

```
#paleta  → display: grid (acá va el layout)
   ├── cajita 1 → fondo de un color + su código HEX
   ├── cajita 2 → ...
   └── ...
```

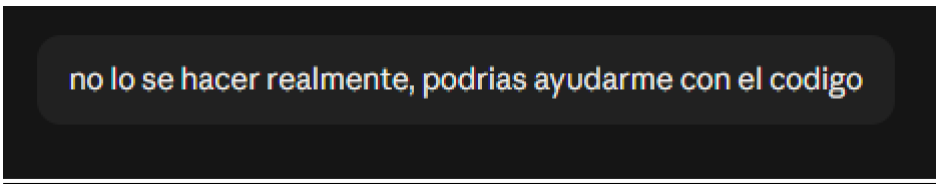
Preguntas para que lo resuelvas

1. ¿Qué propiedad de grid define cuántas columnas hay? Buscá `grid-template-columns` y la función `repeat()`.
2. ¿Y qué propiedad separa las cajitas entre sí? (pista: empieza con `g`).
3. Con 6, 8 y 9 colores, ¿qué cantidad de columnas deja todo prolijo? Probá 3 y 4 en tu cabeza: ¿cuántas filas te quedan y cuántos espacios vacíos?
4. Para que las cajitas tengan altura visible au… e estén vacías, ¿qué propiedad les das?

Lo que decidí o cambié yo: ir armando el contenedor con los conceptos que corresponden, pero vuelvo a tener otra duda y nuevamente consulto.

Consulta: Solicito ayuda para la creación de códigos en CSS.

Prompts:



Respuesta de la IA:

Plantilla con huecos para completar.

Claro, y es normal que se trabe: el selector, el grid y las alturas son cosas nuevas. Como querías hacerlo vos, te doy una **plantilla con huecos** para que completes las partes clave, y yo te doy la solución de lo que se te complique.

1. Pegá esto en tu HTML, dentro de `<section id="paleta">` (es solo de prueba; después se borra):

```html
<div class="color">#FF0000</div>
<div class="color">#00FF00</div>
<div class="color">#0000FF</div>
<div class="color">#FFFF00</div>
<div class="color">#FF00FF</div>
<div class="color">#00FFFF</div>
```

2. En `css/styles.css`**, completá los** `___`**:**

```css
___paleta {
  display: grid;
  grid-template-columns: repeat(___, 1fr);
  ___: 10px;
}

.color {
  ___: 120px;
  background-color: #cccccc;
}
```

Lo que decidí o cambié yo: armar la plantilla y seguir probando, funciona.

Consulta: No puedo ver el botón y evento click, pido que lo revise Claude, luego de varios intentos

Prompts:

necesita encontrar el button id:

```
const boton=document.getElementById("boton");
boton.addEventListener("click", function(){
    console.log("click-");
```

Respuesta de la IA:

¡Buen arranque! Elegiste una forma correcta de empezar: probar con un `console.log` antes de hacer cosas más grandes es una excelente práctica de depuración. Hay dos cosas para corregir.

1. El `id` no coincide con tu HTML

`getElementById("boton")` busca un elemento con `id="boton"`, pero en tu HTML el botón es `<button id="generarBtn">`. Si no lo encuentra, `boton` queda en `null` y la línea siguiente da error. Fijate que el texto entre comillas sea **exactamente** igual al `id` del HTML, con mayúsculas y minúsculas.

2. Te falta cerrar el bloque

Abriste `addEventListener("click", function(){` y esa función tiene que cerrarse, y también el paréntesis de `addEventListener`. Contá lo que abriste:

- `(` de `addEventListener(`
- `{` de `function(){`

Cada uno necesita su cierre, en orden inverso, y la línea termina con `;`. Probá escribir cómo debería verse el final.

Lo que decidí o cambié yo: corregir el id:generarBtn, y cerrar la función que me faltaba. Controlar en la consola que se vea correctamente.

Consulta: sigo sin ver el botón, envío como lo mando y solicito el código para corroborar.

Prompts:

```
const boton=document.getElementById("generarBtn");
boton.addEventListener("click", function(){;
  console.log("click-");
}
```

Respuesta de la IA:

Dale, es un arreglo chico y ya intentaste varias veces. Así queda:

```javascript
const boton = document.getElementById("generarBtn");
boton.addEventListener("click", function () {
    console.log("click-");
});
```

Qué cambió respecto al tuyo

- En la línea 2 saqué el `;` que estaba después de `function(){`.
- En la última línea, `});` hace tres cosas, en este orden: `}` cierra la función, `)` cierra el `addEventListener(` y `;` termina la instrucción.

Lo que decidí o cambié yo: modificar como me dice la IA el código, me faltaban cierre de funciones, espacios.

Consulta: comienzo con JavaScript, sigo con la guía de Claude, genere los botones: click, paletas, cantidad. Aplique funciones y bucle for (estructura), envío el código para que lo revise:

Prompts:

```
boton.addEventListener("click", function () {
  const cantidad = Number(selector.value);
for (let i =0; i < 9; i++) {
  const caja = document.createElement("");
  caja.classList.add("paletas");
  caja.textContent = "Color";
  paleta.appendChild();
}
```

Reviewing the code for hardcoded values and bugs.

Respuesta de la IA:

Vas muy bien: la estructura del bucle `for` está bien armada y los pasos están en el orden correcto. Quedan cuatro detalles.

1. `i < 9` está fijo
Con ese 9, siempre se crean 9 cajitas, elijas lo que elijas. Hace unas líneas guardaste en una variable la cantidad elegida. ¿Qué podés poner ahí en lugar del 9?

2. `document.createElement("")` está vacío
Entre las comillas va el nombre de la etiqueta que querés crear, sin los símbolos `<` y `>`. Fijate qué etiqueta usaste en tus cajitas de prueba del HTML.

3. `paleta.appendChild()` está vacío
Tiene que recibir adentro lo que querés agregar. Mirá la línea donde creaste la cajita: ¿cómo se llama esa variable?

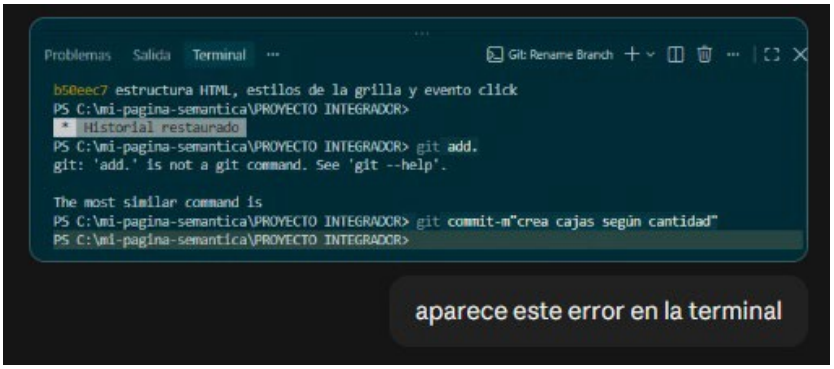
4. La clase `"paletas"`
Revisá tu CSS: ¿existe una regla `.paletas`? La clase que le da altura a las cajitas era otra. Si la clase no existe en el CSS, las cajitas van a estar ahí, pero sin altura visible.

5. Falta cerrar el final
Contá lo que abriste:

Lo que decidí o cambié yo: modificar variables, etiquetas, clases.

Consulta: Creación del repositorio en GitHub y subir los commits.

Prompts:



Respuesta de la IA: Una comparación de lo que escribí vs lo que debería ser.

Lo que escribiste vs. lo que necesita Git

Escribiste	Tiene que ser
`git add.`	`git add .` (espacio entre `add` y el punto)
`git commit-m"crea cajas..."`	`git commit -m "crea cajas..."` (espacio antes de `-m` y antes de las comillas)

Con `git add.`, Git leyó `add.` como un comando inventado y por eso dijo `'add.' is not a git command`. Con `git commit-m"..."` pasó lo mismo, así que **ninguno de los dos hizo nada**.

Lo que decidí o cambié yo: modificación de espacios, comillas, luego cargarlo en la terminal como corresponde.

Consulta: Probé el for y cuando elegía 8 colores aparecían 14 cajitas. Entendí que se sumaban, pero no sabía cómo limpiarlas. Claude me explicó la idea del borrador con innerHTML, pero me costó ubicar la línea, cuando me dijo el código lo entendí.

en mi resumen de que significa innerhtml no puedo entender como aplicarlo dentro

Respuesta de la IA: Explicación de innerHtml y ejemplo:

`innerHTML` es una propiedad de un elemento que representa **todo lo que tiene adentro**. Podés **leerla** (ver qué hay) o **asignarle un valor** (reemplazar lo que hay por eso otro).

Un ejemplo distinto al tuyo

Imaginá este HTML:

```html
<div id="caja">
    <p>Hola</p>
    <p>Chau</p>
</div>
```

Y este JavaScript:

```javascript
const caja = document.getElementById("caja");
caja.innerHTML = "";
```

Se lee: "lo que hay adentro de `caja` ahora es... nada". Los dos `<p>` desaparecen y `caja` queda vacía. Las comillas sin nada entre ellas son un texto vacío.

Otro ejemplo, con otra cosa

Una lista de compras en el HTML:

```html
<ul id="lista"></ul>
```

```javascript
const lista = document.getElementById("lista");

lista.innerHTML = "";          // 1. pasar el borrador

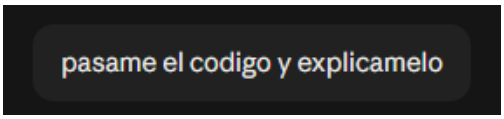
for (let i = 0; i < 3; i++) {  // 2. escribir lo nuevo
    const item = document.createElement("li");
    item.textContent = "producto";
    lista.appendChild(item);
}
```

Fijate el **orden**: primero el borrador (línea 1), después el bucle que escribe (línea 2). Si el borrador estuviera adentro del bucle, borraría cada producto justo después de escribirlo, y solo quedaría el último.

Lo que decidí o cambié yo: probar distintas maneras en VSS hasta que tuve que pedir el código porque no sabía aplicarlo.

Consulta: pasarme el código para ponerlo en mi JavaScript, luego de varios intentos.

Prompts:



Respuesta de la IA: me pasa el código y la explicación paso por paso



Lo que decidí o cambié yo: entendí que el innerHTML va antes del for, y afuera del bucle, porque adentro borraría cada caja recién creada. Lo probe con 6, 8 y 9, y siempre quedo la cantidad elegida.

,

Consulta: creación de colores aleatorios en HSL.

Prompts:

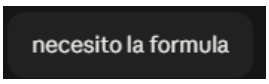
```
const boton = document.getElementById("generarBtn");
const selector= document.getElementById("cantidad");
const paleta = document.getElementById("paleta");


boton.addEventListener("click", function () {
    const cantidad = Number(selector.value);

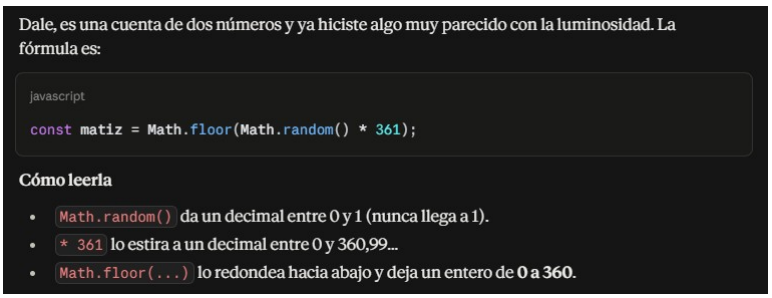
paleta.innerHTML ="";

    for (let i = 0; i < cantidad; i++) {
        const caja = document.createElement("div");
        caja.classList.add("color");
        caja.textContent = "Color";
        caja.style.backgroundColor = hsl (30, 80%, 50%);
        paleta.appendChild(caja);
    }
});
```





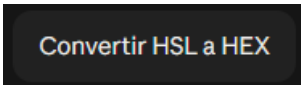
Respuesta de la IA: varias correcciones de código: el HSL(...) tiene que ser un texto entre comillas, y para meter variables hay que usar comillas invertidas y ${}. Mi fórmula de luminosidad estaba bien armada. El matiz seguía fijo (primero en 30 y después en 358), así que todas las cajitas tenían el mismo color. Mi rango de matiz (20 a 44) solo daba naranjas. Me mostró la tabla de matices y cómo armar la fórmula con otro rango. Como no me salía, me dio la fórmula completa del matiz.



Lo que decidí o cambié yo: corregir todo lo que me dijo la IA y probarlo cargando la página: corregí el HSL para que fuera un texto con comillas invertidas y ${}. Probe un rango de matiz de 20 a 44 y me aparecía solamente el color naranja, asi que lo cambie por la formula de 0 a 360. Dejé la luminosidad entre 40% y 80% y declaré las variables adentro del for para que cada cajita tenga su propio color

Consulta: Revisé el material de lectura y no encontré cómo hacer la conversión. Le pedí ayuda a Claude para entender cómo convertir HSL a HEX en lugar de generar el color directo en HEX.

Prompts:



pero tengo paletas aleatorias, son varias como hago para ver que color es y que codigo le corresponde

en el paso 1 con (255).toString(16) (10).toString(16) me devuelve una cajita con distintos colores, sin mostrar el codigo

ayuda con el codigo

Respuesta de la IA: Explicación de que es un HEX, me dio las funciones ya hechas aHex, que pasa un número a base 16 usando padStart, y hslAHex, que es un algoritmo estándar de conversión. Al final, como no sabía conectar la función con mi for, me dio el bloque completo con la variable hex.

Qué es un HEX

`#RRGGBB` son tres números (rojo, verde, azul), cada uno entre 0 y 255, escritos en **base 16** y con dos caracteres cada uno. Por ejemplo, 255 en base 16 es `ff`, y 10 es `0a` (con el 0 adelante).

Parte 1: pasar un número a dos caracteres hexadecimales

Probá esto en la consola del navegador (F12 → Console):

```javascript
(255).toString(16)
(10).toString(16)
```

- ¿Qué devuelve cada uno? Fijate que el segundo da un solo carácter y necesitamos siempre dos.
- Investigá `padStart` en MDN: sirve para rellenar un texto hasta cierto largo.
- Con eso armá una pequeña función `aHex(numero)` que devuelva siempre dos caracteres.

Parte 2: pasar de HSL a RGB

Es la parte con matemática. La fórmula es conocida y no hace falta inventarla: buscá "HSL to RGB algorithm" o la sección de conversión de la especificación CSS Color. Antes de escribir código, ubicá estas cosas:

- Qué entradas necesita: matiz (0–360), saturación y luminosidad (en la fórmula suelen usarse como decimales de 0 a 1, no como porcentajes).
- Qué devuelve: tres números, rojo, verde y azul (0–255).
- Cuáles son sus pasos, para pasarlos a JavaScript de a poquito.

Tip: escribí las instrucciones en español, línea por línea, antes de convertirlas en código.

Parte 3: juntar todo en una función

```javascript
function hslAHex(h, s, l) {
    // 1. convertir a rojo, verde y azul
    // 2. pasar cada uno a dos caracteres hex
    // 3. devolver "#" + los tres juntos
}
```

Dale. La conversión de HSL a RGB es un algoritmo estándar, no algo que haya que inventar, así que te paso las dos funciones y vos las conectás con tu proyecto.

Funciones: van arriba de todo en `app.js`, afuera del `addEventListener`

```javascript
function aHex(numero) {
    return numero.toString(16).padStart(2, "0");
}

function hslAHex(h, s, l) {
    s = s / 100;
    l = l / 100;
    const a = s * Math.min(l, 1 - l);

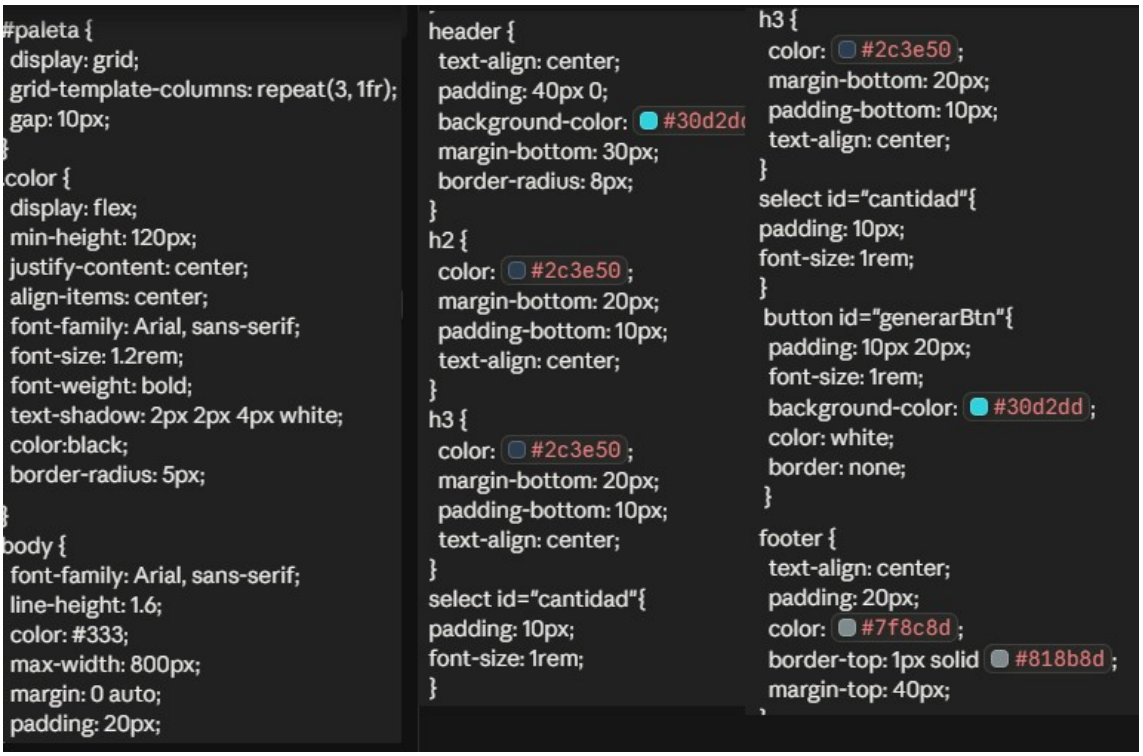
    function canal(n) {
        const k = (n + h / 30) % 12;
        const valor = l - a * Math.max(Math.min(k - 3, 9 - k, 1), -1);
        return Math.round(255 * valor);
    }
```

Lo que decidí o cambié yo: Decidí reemplazar padStart por un if, porque no lo vimos en el curso. En mi primera versión tenía dos errores, toString(1) en vez de toString(16) y lenght en vez de length, y los corregí siguiendo las pistas. También moví las funciones afuera del addEventListener. Entiendo que aHex agrega un "0" cuando el número tiene un solo carácter.

Consulta: Ahora que termine con la generación de paletas, le voy a dar estilo a mi css.

Prompts:

- Aplicar principios básicos de diseño visual mediante CSS. Evitar el uso de estilos inline y vincular correctamente hojas de estilo externas.

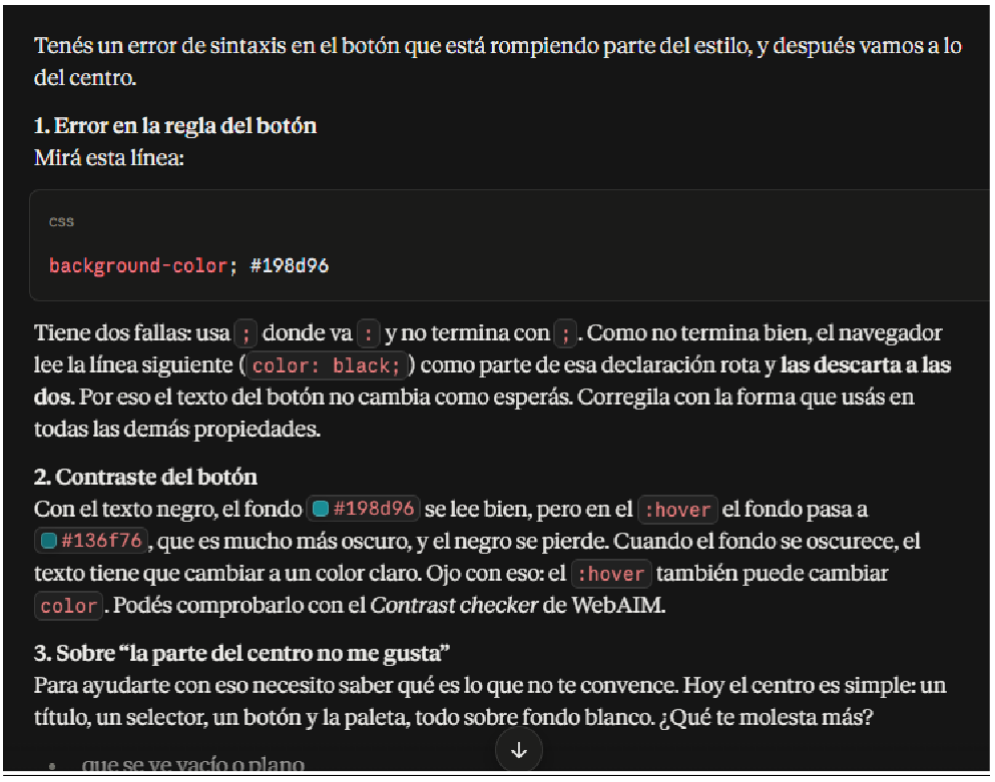


```
#paleta {
 display: grid;
 grid-template-columns: repeat(3, 1fr);
 gap: 10px;
}
color {
 display: flex;
 min-height: 120px;
 justify-content: center;
 align-items: center;
 font-family: Arial, sans-serif;
 font-size: 1.2rem;
 font-weight: bold;
 text-shadow: 2px 2px 4px white;
 color:black;
 border-radius: 5px;
}
body {
 font-family: Arial, sans-serif;
 line-height: 1.6;
 color: #333;
 max-width: 800px;
 margin: 0 auto;
 padding: 20px;
```

```
header {
 text-align: center;
 padding: 40px 0;
 background-color: #30d2d
 margin-bottom: 30px;
 border-radius: 8px;
}
h2 {
 color: #2c3e50;
 margin-bottom: 20px;
 padding-bottom: 10px;
 text-align: center;
}
h3 {
 color: #2c3e50;
 margin-bottom: 20px;
 padding-bottom: 10px;
 text-align: center;
}
select id="cantidad"{
padding: 10px;
font-size: 1rem;
}
```

```
h3 {
 color: #2c3e50;
 margin-bottom: 20px;
 padding-bottom: 10px;
 text-align: center;
}
select id="cantidad"{
padding: 10px;
font-size: 1rem;
}
 button id="generarBtn"{
 padding: 10px 20px;
 font-size: 1rem;
 background-color: #30d2dd;
 color: white;
 border: none;
 }
footer {
 text-align: center;
 padding: 20px;
 color: #7f8c8d;
 border-top: 1px solid #818b8d;
 margin-top: 40px;
```

Respuesta de la IA: Corrección de sintaxis, contrastes y que el botón se note como clickeable.



Tenés un error de sintaxis en el botón que está rompiendo parte del estilo, y después vamos a lo del centro.

1. Error en la regla del botón
Mirá esta línea:

```
css
background-color; #198d96
```

Tiene dos fallas: usa `;` donde va `:` y no termina con `;`. Como no termina bien, el navegador lee la línea siguiente (`color: black;`) como parte de esa declaración rota y **las descarta a las dos**. Por eso el texto del botón no cambia como esperás. Corregila con la forma que usás en todas las demás propiedades.

2. Contraste del botón
Con el texto negro, el fondo `#198d96` se lee bien, pero en el `:hover` el fondo pasa a `#136f76`, que es mucho más oscuro, y el negro se pierde. Cuando el fondo se oscurece, el texto tiene que cambiar a un color claro. Ojo con eso: el `:hover` también puede cambiar `color`. Podés comprobarlo con el *Contrast checker* de WebAIM.

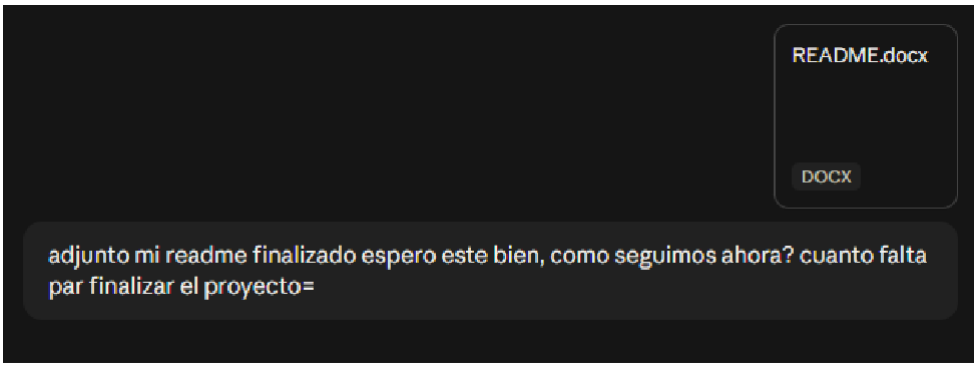
3. Sobre "la parte del centro no me gusta"
Para ayudarte con eso necesito saber qué es lo que no te convence. Hoy el centro es simple: un título, un selector, un botón y la paleta, todo sobre fondo blanco. ¿Qué te molesta más?

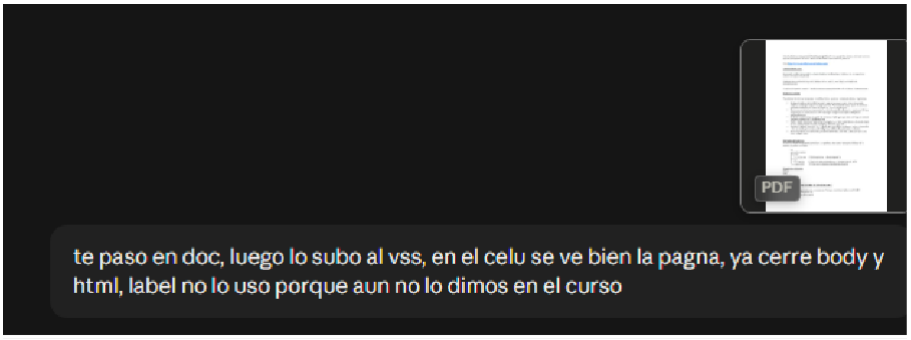
- que se ve vacío o plano

Lo que decidí o cambié yo: cambie el contraste, le agregue para que se pueda ver bien en un celular, arregle la sintaxis de lo que me rompía el estilo.

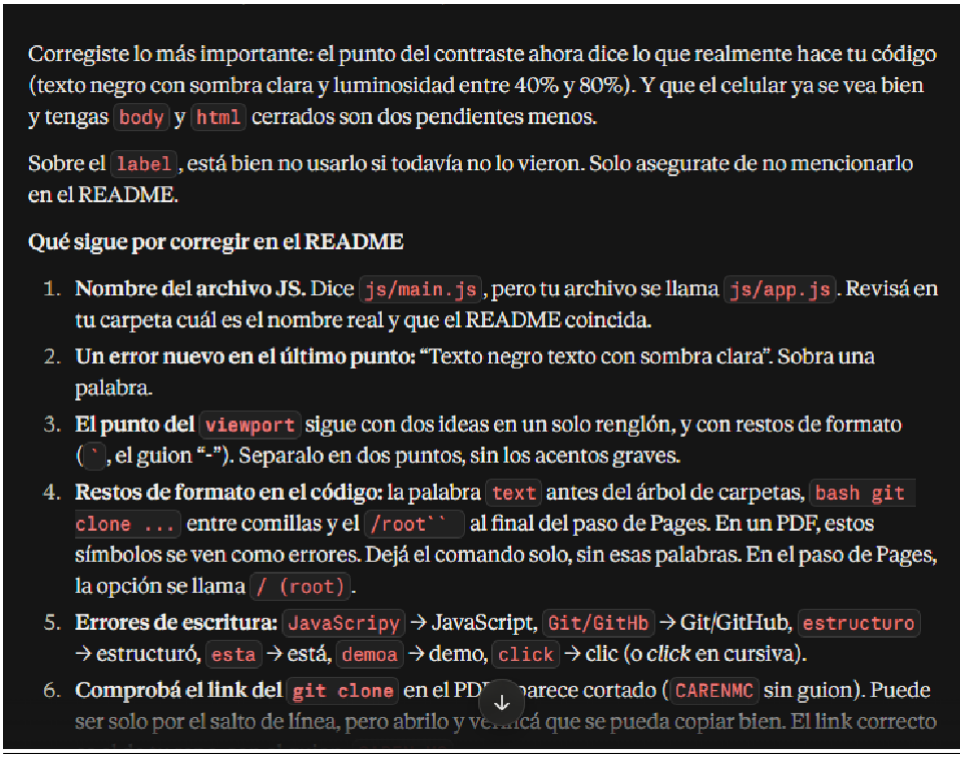
Consulta: Corrección del README

Prompts:





Respuesta de la IA:



Corregiste lo más importante: el punto del contraste ahora dice lo que realmente hace tu código (texto negro con sombra clara y luminosidad entre 40% y 80%). Y que el celular ya se vea bien y tengas `body` y `html` cerrados son dos pendientes menos.

Sobre el `label`, está bien no usarlo si todavía no lo vieron. Solo asegurate de no mencionarlo en el README.

Qué sigue por corregir en el README

1. **Nombre del archivo JS.** Dice `js/main.js`, pero tu archivo se llama `js/app.js`. Revisá en tu carpeta cuál es el nombre real y que el README coincida.
2. **Un error nuevo en el último punto:** "Texto negro texto con sombra clara". Sobra una palabra.
3. **El punto del** `viewport` sigue con dos ideas en un solo renglón, y con restos de formato (`, el guion "-"). Separalo en dos puntos, sin los acentos graves.
4. **Restos de formato en el código:** la palabra `text` antes del árbol de carpetas, `bash git clone ...` entre comillas y el `/root`` al final del paso de Pages. En un PDF, estos símbolos se ven como errores. Dejá el comando solo, sin esas palabras. En el paso de Pages, la opción se llama `/ (root)`.
5. **Errores de escritura:** `JavaScripy` → JavaScript, `Git/GitHb` → Git/GitHub, `estructuro` → estructuró, `esta` → está, `demoa` → demo, `click` → clic (o *click* en cursiva).
6. **Comprobá el link del** `git clone` en el PDF, parece cortado (`CARENMC` sin guion). Puede ser solo por el salto de línea, pero abrilo y verificá que se pueda copiar bien. El link correcto

Lo que decidí o cambié yo: modificación de nombre de carpeta, errores de escritura, lo modifique y cargue en GitHub.

Consulta: tengo que agregar el formato HSL en mi JavaScript, solo muestra formato HEX.

Prompts:

> en el HTML le deberia agregar <option value>hex o hsl, con el select id formato? en JavaScript, deberia agregar const formato?? y luego modifico el for?

Respuesta de la IA:

Me dijo que estaba bien encaminado, pero que me faltaba otra línea mas y que vea el if y el else.

1. HTML

Un segundo `<select>` con `id="formato"`, copiando la forma del que ya tenés para la cantidad. Un detalle: en cada `<option>`, el `value` va **adentro de la etiqueta de apertura**, con comillas, y el texto que ve la persona va **entre la apertura y el cierre**. Con otro ejemplo:

```html
<option value="valor-interno">Texto que se ve</option>
```

Para tus opciones, usá los valores `hex` y `hsl` en minúscula, que son los que después vas a comparar en JavaScript. Ubicalo junto al selector de cantidad, antes del botón.

2. JavaScript

Sí, necesitás una constante nueva, y va arriba con las otras tres, porque busca un elemento del HTML igual que ellas:

```javascript
const formato = document.getElementById("___");
```

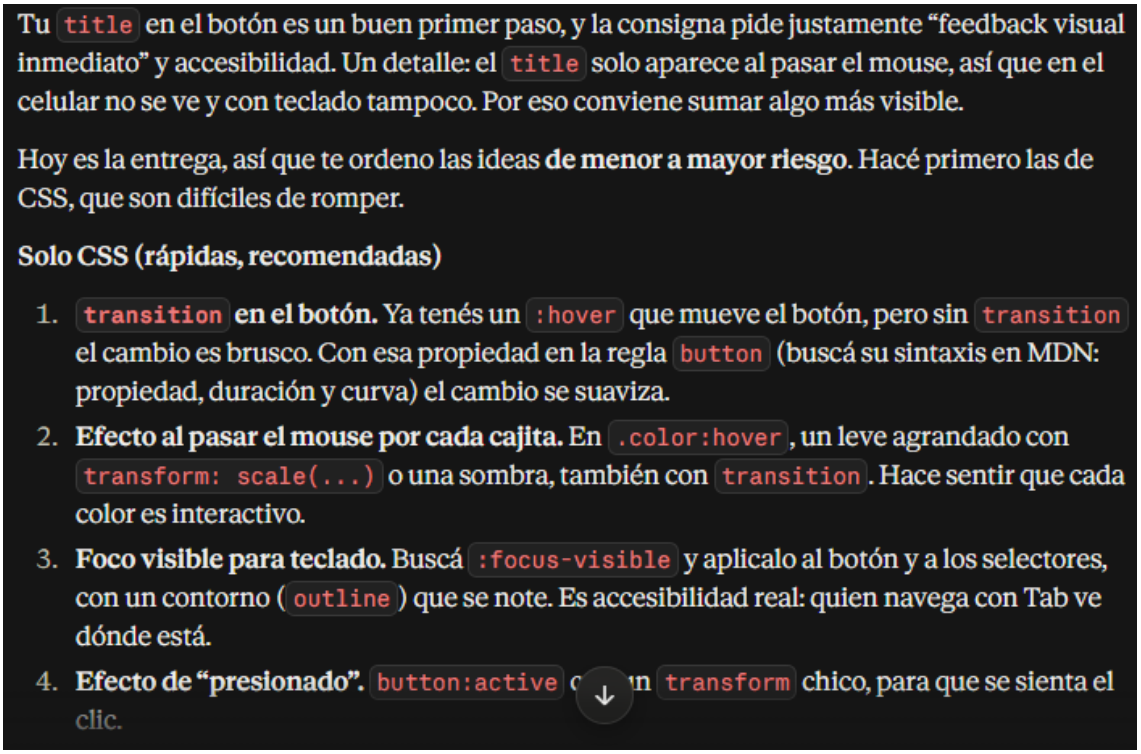
Lo que decidí o cambié yo: agrego la constante formato arriba junto con las demás constantes y agrego también la const hsl dentro del for, luego sumo if y else para la selección de un formato u otro.

Consulta: debo agregar microfeedbacks a mi proyecto, como hacerlo?

Prompts:

> necesito generar microfeedbacks a mi proyecto, utilice solamente en mi html, en el buttonid, un title que dice generar paleta nueva, me ayudas con otra idea?

Respuesta de la IA: Ideas de como aplicarlos dentro de mi proyecto.



Tu `title` en el botón es un buen primer paso, y la consigna pide justamente "feedback visual inmediato" y accesibilidad. Un detalle: el `title` solo aparece al pasar el mouse, así que en el celular no se ve y con teclado tampoco. Por eso conviene sumar algo más visible.

Hoy es la entrega, así que te ordeno las ideas **de menor a mayor riesgo**. Hacé primero las de CSS, que son difíciles de romper.

Solo CSS (rápidas, recomendadas)

1. `transition` **en el botón.** Ya tenés un `:hover` que mueve el botón, pero sin `transition` el cambio es brusco. Con esa propiedad en la regla `button` (buscá su sintaxis en MDN: propiedad, duración y curva) el cambio se suaviza.
2. **Efecto al pasar el mouse por cada cajita.** En `.color:hover`, un leve agrandado con `transform: scale(...)` o una sombra, también con `transition`. Hace sentir que cada color es interactivo.
3. **Foco visible para teclado.** Buscá `:focus-visible` y aplicalo al botón y a los selectores, con un contorno (`outline`) que se note. Es accesibilidad real: quien navega con Tab ve dónde está.
4. **Efecto de "presionado".** `button:active` c…n `transform` chico, para que se sienta el clic.

<u>Lo que decidí o cambié yo</u>: agregar la propiedad transition que hace que un cambio tarde unos segundos y se vea suavemente dentro del button.